



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Евалуација различитих софтверских имплементација машина стања на ресурсно ограниченим системима

Нови Сад, 2026. године

Садржај

1. Увод.....	5
2. Теоријске основе.....	8
2.1 Угњеждена <i>Switch-case FSM</i>	8
2.2 Модификована угњеждена <i>Switch-case FSM</i>	10
2.3 <i>FSM</i> структурног низа.....	11
2.4 Директни <i>LUT FSM</i>	13
2.5 <i>FSM</i> објеката стања.....	14
2.6 Планарни <i>HSM FSM</i>	16
2.8 Угњеждени <i>HSM FSM</i>	19
2.7 Угњеждени <i>LUT FSM</i> (Santic).....	21
3. Методологија мерења.....	23
3.1 Заједничка машина стања.....	23
3.2 Хардверске платформе.....	25
3.3 Методологија мерења циклуса.....	26
3.3.1 Зашто није коришћен <i>ISR</i> приступ?.....	26
3.3.2 <i>Polling</i> приступ коришћен за мерење.....	27
3.3.3 Регистри <i>Timer1</i> коришћени за мерење.....	28
3.3.4 Заједничка функција за мерење транзиције.....	29
3.4 Нивои оптимизације компајлера.....	30
4. Резултати мерења и дискусија резултата.....	31
4.1 Резултати на <i>AVR</i> платформи.....	31
4.1.1 Разлика између стилова писања кода – <i>return</i> наспрам <i>break</i>	33
4.1.2 Директни <i>LUT FSM</i> — потврда константне временске сложености.....	33
4.1.3 <i>FSM</i> објеката стања — компромис меморије и брзине.....	34
4.1.4 Хијерархијске имплементације — најскупљи приступ на сваком нивоу	34
4.1.5 Индиректни <i>LUT FSM</i> (Santic) — трошак провере граница.....	34
4.2 Поређење <i>AVR</i> резултата са литературом.....	34
4.3 Резултати на <i>ESP32</i> платформи.....	35
4.4 Упоредна анализа <i>AVR</i> и <i>ESP32</i>	35
4.5 Разлагање трошка индиректних приступа.....	35
Литература.....	36

1. Увод

Машине стања (*Finite State Machine, FSM*) представљају модел понашања система заснован на коначном скупу стања. Прелазак из једног стања у друго дешава се према унапред дефинисаним правилима, у зависности од догађаја који се јаве у систему. Ова техника нашла је примену у широком спектру области попут комуникационих протокола, системима управљања, графичким корисничким интерфејсима. Посебно је значајна у развоју софтвера за уграђене системе (*embedded systems*) — рачунарске системе уграђене унутар неког уређаја, најчешће засноване на микроконтролеру, наменски пројектоване да обављају одређену, ограничену функцију уз строга ограничења у погледу расположиве меморије, процесорске снаге и потрошње енергије. На оваквим системима, јасно дефинисана и предвидива логика коју машина стања пружа олакшава пројектовање поузданог понашања упркос ограниченим ресурсима. Иако је концепт машине стања теоријски добро утемељен, начин на који се она преводи у изворни код програмског језика није јединствен. Исти проблем може бити решен на неколико суштински различитих начина, при чему избор између њих није увек очигледан, нити подједнако документован у литератури.

John Santic то прецизно формулише на почетку свог рада: прво се идентификују сва могућа стања система, затим сви догађаји који могу изазвати промену тог стања или извршавање неке акције и на крају скуп акција које систем предузима за сваку комбинацију стања и догађаја. Santic, такође наводи и да постоје два уобичајена начина да се машина стања представи у C програмском језику: преко угњеждених *switch* исказа, или преко табеле претраге (*lookup table*). Табеле претраге карактерише као концизнији начин, али који, за разлику од *switch-a*, нема уграђену заштиту за неважеће комбинације стања и догађаја, па се граница мора проверити ручно [JSnd] .

Мотивацију за полазну тачку овог рада представља запажање, да два наизглед различита начина записивања исте логике носе различите практичне последице. Чак ни аутор Santic који их директно упоређује не даје мерљив, квантификован одговор о томе колико та разлика заправо кошта. Ако постоји мерљива разлика између само два приступа које Santic пореди, поставља се питање да ли постоји слична или израженија разлика између знатно већег броја имплементационих приступа доступних у пракси и у литератури. Ови приступи обухватају угњеждени *switch* у различитим стилским варијантама, структурни низ, директне и индиректне табеле претраге, низове функција по стању и хијерархијску организацију стања. То питање додатно је поткрепљено и запажањем из академске литературе у раду који пореди шест различитих архитектонских приступа имплементацији машине стања у C-у. У том раду, аутори Carlgren и Oskarsson експлицитно приказују резултате заузећа меморије и времена извршавања одвојено, за случај без оптимизационих флегова и за случај са -O2 флегом. Аутори су увидели да се рангирање приступа по меморијској ефикасности битно разликује у зависности од примењене оптимизације [CO23] . Ово је сугерисало да поређење

имплементационих приступа мора узети у обзир и ниво оптимизације преводиоца, не само сам архитектонски избор.

Постојећа литература о начинима пројектовања машина стања — Adamczyk, Carlgren и Oskarsson, између осталих — углавном даје теоријску, квалитативну оцену сложености појединих приступа, без систематског, упоредивог емпиријског мерења на конкретном, ресурсно ограниченом хардверу.

Овај рад полази управо одатле: уместо да имплементациони приступи остану упоређени само на нивоу теорије, сваки од њих је имплементиран, измерен и упоређен на стварним микроконтролерским платформама, чиме се теоријске тврдње из литературе емпиријски проверавају, квантификују, и, где је то случај, допуњују или оспоравају.

Предмет овог рада је поређење осам имплементационих приступа машини стања, за сваки од којих је у наставку рада уведен и доследно коришћен назив на српском језику, уз оригинални (енглески) назив наведен у загради при првом помињању:

1. Угњеждена *Switch-case FSM (Nested Switch — break)*
2. Модификована угњеждена *Switch-case FSM (Nested Switch — return)*
3. *FSM* структурног низа (*Array of Structs*)
4. Директни *LUT FSM (Indexed Table)*
5. *FSM* објекта стања (*State Object*)
6. Планарни *HSM FSM (HSM flat)*
7. Угњеждени *HSM FSM (HSM nested)*
8. Индиректни *LUT FSM (Santic Lookup Table)*

Сваки од ових приступа имплементиран је над идентичним функционалним аутоматом и измерен на **четири** архитектонски различите платформе:

ATmega328P (Arduino Uno, 8-битна *AVR* архитектура) и *ESP32* (32-битна *Xtensa* архитектура), *Raspberry Pi 4* (64-битна *ARM Cortex-A72* архитектура, под управљањем *Linux* оперативног система), и класичан рачунар опште намене (*x86_64* архитектура, под управљањем *Linux* оперативног система).

За сваку имплементацију мерено је заузеће програмске и радне меморије, као и број процесорских циклуса по транзицији, на више нивоа оптимизације преводиоца. Добијени резултати упоређени су са теоријским тврдњама из релевантне литературе, чиме се утврђује у којој мери се теоријски очекивано понашање поклапа са стварно измереним.

Резултати овог рада релевантни су у ширем контексту развоја софтвера за уграђене системе, чији значај континуирано расте са ширењем интернета ствари (*Internet of Things - IoT*) и порастом броја уређаја који, упркос ограниченим ресурсима, поуздано морају да извршавају све сложенију управљачку логику.

Резултати рада пружају конкретну основу за избор имплементационог приступа машина стања, како избор имплементације не би био заснован на интуицији или стилу, већ на експерименталним резултатима који приказују компромисе између брзине, меморије и прегледности кода.

Рад је организован на следећи начин. Поглавље 2 даје теоријски преглед испитиваних имплементационих приступа машини стања. Поглавље 3 описује методологију мерења и коришћени хардвер. Поглавље 4 представља измерене резултате и њихову анализу, укључујући поређење са теоријским очекивањима из литературе. Поглавље 5 садржи закључке рада и предлоге за даљи рад.

2. Теоријске основе

У наставку је дат теоријски преглед свих осам имплементационих приступа машина стања наведених у Уводу. За сваки приступ наведена је релевантна литература која га описује — механизам рада, теоријска оцена сложености, као и познате предности и мане. За сваки од наведених приступа, посебно је размотрено да ли имплементација коришћена у раду верно одговара концепту из цитираних извора или се увиђају било каква одступања. На местима где одступање постоји, то је експлицитно наведено и образложено. Овакав приступ обезбеђује да поређење са резултатима мерења у Поглављу 4 буде прецизно утемељено на ономе што је стварно имплементирано, а не на претпоставци да свака имплементација доследно прати свој извор до најситнијег детаља.

2.1 Угњеждена *Switch-case FSM*

Угњеждени *switch* искази (*nested switch statements*) представљају један од најстаријих и најраспрострањенијих приступа имплементацији машина стања. Adamczyk их у уводном делу свог рада, уз каскадне *if* исказе, сврстава међу најпопуларније не-објектно-оријентисане имплементације *FSM*-а. Овај приступ је у изузетно честој употреби и уз то обезбеђује брзо извршавање, по цени слабије читљивости и одрживости кода.

Carlgren и Oskarsson дају прецизнији структурни опис приступа: спољашњи *switch* исказ прати тренутно стање као скаларну променљиву, док унутрашњи *switch* исказ прати тренутни догађај, такође као скаларну променљиву. Аутори истичу да је ово ефикасно решење за једноставне, планарне (*flat*) аутомате, али без подршке за напреднију функционалност попут хијерархије или историје стања. Конкретан пример кода који аутори прилажу (Слика 1) генерише имплементацију у којој се резултат транзиције прво уписује у привремену променљиву, а затим се излази из оба нивоа *switch*-а путем *break* исказа.

```
1  ...
2  switch(state)
3  {
4      case StateA:
5      {
6          switch(event)
7          {
8              case Event1:
9              {
10                 nextState = Event1Handler();
11                 break;
12             }
13         }
14         break;
15     }
16     case StateB:
17     ...
```

Слика 1- Carlgren и Oskarsson угњеждени *switch-case* [CO23]

Ова структура се поклапа са имплементацијом коришћеном у овом раду (Кодни листинг 1). Постоји једна разлика: њихов генерисани код сваку транзицију делегира одвојеној *eventHandler* функцији, која у основном облику само враћа следеће стање, али представља место предвиђено за додатну корисничку логику за акцију. Имплементација у овом раду *next_state* уписује директно унутар *case* гране, без тог посредничког позива функције. Овим се избегава додатни трошак позива функције који није предмет мерења. Исту структуру диспечовања (начин на који програм, када добије тренутно стање и тренутни догађај, долази до одлуке која транзиција треба да се изврши) — *switch(CurrentState)* са унутрашњим грањањем по догађају — користе и Kadam, Jogalekar и Nembade у свом *FSM2Construct* алгоритму [KJH23], што потврђује да се ради о општеприхваћеном приступу, а не о решењу специфичном за један извор.

Ниједан од наведених извора не даје формалну оцену временске сложености угњежденог *switch* приступа (за разлику од *State Table Pattern-a*, за који Adamczyk експлицитно наводи $O(c)$). Очекивано понашање зависи од тога да ли компајлер успева да густо попуњен *switch* преведе у табелу скокова (ефективно $O(1)$) или у ланац узастопних поређења ($O(\text{број грана})$). Carlgren и Oskarsson емпиријски показују да Nested Switch приступ при

-O2 нивоу оптимизације захтева најмање програмске меморије од свих испитаних приступа, уз напомену да његова читљивост опада са порастом броја стања и догађаја услед све дубље угњеждености [CO23].

```
switch (state) {
    case STATE_IDLE:
        switch (event) {
            case EV_VALID:
                next_state = STATE_CHECKING;
                break;
            default:
                next_state = STATE_IDLE;
                break;
        }
        break;
    case STATE_CHECKING:
        switch (event) {
            case EV_VALID:
                next_state = STATE_GRANTED;
                break;
            case EV_INVALID:
                next_state = STATE_DENIED;
                break;
            ...
        }
    return next_state;
```

Кодни листинг 1 - Угњеждена Switch-case FSM

2.2 Модификована угњеждена *Switch-case FSM*

Имплементација модификованог угњеженог *Switch-case FSM*-а задржава идентичан механизам диспечовања описан у претходном одељку (спољашњи *switch* по стању, унутрашњи по догађају), уз једну измену: резултат транзиције се враћа директно из сваке гране (*return STATE_X;*), уместо да се прво упише у привремену променљиву па затим врати коришћењем *break* исказа на крају функције.

За разлику од *break* варијанте, која се доследно поклапа са конкретним примером кода (Слика 1) који Carlgren и Oskarsson прилажу у свом раду, *return* варијанта не одговара доследно ниједном од цитираних извора — представља проширење уведено у овом раду (Кодни листинг 2), ради изолованог мерења утицаја стила писања кода на перформансе, независно од саме алгоритамске структуре диспечовања која остаје идентична *break* варијанти.

Будући да су обе варијанте семантички еквивалентне (кодирају идентичну транзициону табелу, само различитим синтаксним обрасцем). Свака измерена разлика у броју циклуса или заузећу меморије између њих може се приписати искључиво начину писања кода, а не разлици у самом алгоритму, што ову варијанту чини контролном тачком за проверу да ли, и у којој мери, стилски избор при писању кода утиче на перформансе, питање које ниједан од цитираних извора директно не разматра.

```
...
switch (state) {
    case STATE_IDLE:
        switch (event) {
            case EV_VALID:
                return STATE_CHECKING;
            default:
                return STATE_IDLE;
        }
    case STATE_CHECKING:
        ...
}
```

Кодни листинг 2 – Модификована угњеждена Switch-case FSM

2.3 FSM структурног низа

Carlgren и Oskarsson описују *FSM* структурног низа (*Array of Structs* приступ) као алтернативу угнеженом *switch*-у за имплементацију планарних (*flat*) аутомата: генерише се низ структура које садрже све валидне комбинације стања и догађаја, а индексирање се заснива на енумерисаним типовима [CO23]. Аутори наводе да је њихов приступ заснован на чланку Amlendra Kumara, који први демонстрира ову технику кроз пример имплементације аутомата банкомата у C-у, [AK17]. Структура Kumar-овог кода, као и структура коју генеришу Carlgren и Oskarsson (Слика 2), садржи показивач на *eventHandler* функцију као треће поље структуре — {стање, догађај, показивач на *handler*}. Након што линеарна претрага пронађе одговарајући унос, следеће стање се добија позивом те функције, а не директним читањем поља из структуре.

```
1  ...
2  typedef struct
3  {
4      enumStates stateMachineState;
5      enumEvents stateMachineEvent;
6      eventHandler stateMachineEventHandler;
7  } stateMachineStruct;
8  ...
9  stateMachineStruct stateMachine[] =
10 {
11     {StateA, Event1, Event1Handler},
12     {StateA, Event2, Event2Handler}
13 };
14 ...
15 nextState = (*stateMachine[newEvent].stateMachineEventHandler)();
16 ...
```

Слика 2 - Carlgren i Oskarsson array of structs implementation [CO23]

У овом раду имплементиране су две варијанте овог приступа.

Прва варијанта у потпуности одговара и Kumar-овом приступу (Слика 2) која садржи позив `transition_table[i].handler()`, чиме доследно прати изворни концепт (Кодни листинг 3). Друга, поједностављена варијанта коју уводи овај рад, у трећем пољу структуре директно чува вредност следећег стања (Кодни листинг 4), без посредничког позива функције. Ова поједностављена варијанта не одговара ниједном од цитираних извора и уведена је као контролна тачка, како би се приказао допринос појединачне компоненте (позива функције преко показивача) у трошку укупне транзиције, независно од трошка саме линеарне претраге, који је идентичан у обе имплементације.

Будући да је линеарна претрага идентична у обе имплементације, разлика у броју циклуса између њих треба да одговара тачно трошку једног индиректног позива функције — константном, малом износу независном од дужине табеле.

```

typedef struct {
    uint8_t state;
    uint8_t event;
    EventHandler handler; // pokazivac na funkciju, ne
    next_state konstanta
} Transition;

static const Transition transition_table[] = {
    { STATE_IDLE,      EV_VALID,    handler_idle_valid },
    { STATE_CHECKING,  EV_VALID,    handler_checking_valid },
    { STATE_CHECKING,  EV_INVALID,  handler_checking_invalid },
    { STATE_GRANTED,   EV_TIMEOUT,  handler_granted_timeout },
    { STATE_DENIED,    EV_TIMEOUT,  handler_denied_timeout },
};

```

Кодни листинг 3 – структурни низ sa handler функцијом

```

typedef struct {
    uint8_t state;
    uint8_t event;
    uint8_t next_state;
} Transition;

static const Transition transition_table[] = {
    { STATE_IDLE,      EV_VALID,    STATE_CHECKING },
    { STATE_CHECKING,  EV_VALID,    STATE_GRANTED },
    { STATE_CHECKING,  EV_INVALID,  STATE_DENIED },
    { STATE_GRANTED,   EV_TIMEOUT,  STATE_IDLE },
    { STATE_DENIED,    EV_TIMEOUT,  STATE_IDLE },
};

```

Кодни листинг 4- структурни низ без handler функције

Поређење две имплементације, које се разликују искључиво по томе да ли се следеће стање чита директно из поља структуре или се добија позивом *eventHandler* функције преко показивача (образац који одговара и Kumaг-овом оригиналном коду и генерисаном коду Carlgren-a и Oskarsson-a, Слика 2), показује на AVR платформи константну, малу разлику у броју циклуса: 12 циклуса на -Os и -O2, и 18 циклуса на -O0. Пошто је линеарна претрага кроз табелу транзиција идентична у обе имплементације, ова разлика произилази искључиво од трошка самог индиректног позива функције. За разлику од стилске разлике између *return* и *break*, имплементација угњежженог *switch-a*, која при оптимизацији у потпуности нестаје, ова разлика остаје присутна и на -O2 зато што компајлер не може унапред да зна која се конкретна функција позива преко показивача, па не може да изврши уметање тог позива.

Као представник *FSM структурног низа* приступа, у наставку рада коришћена је поједностављена варијанта, без посредничког позива *eventHandler* функције, што значи да се следеће стање директно чита из структуре.

Handler функција у оригиналном концепту [AK17] , [CO23] , постоји да би извршила неку акцију уз транзицију, док је стање само њена повратна вредност. Пошто FSM у овом раду не имплементира никакву додатну акцију у тим функцијама се мери искључиво сама транзиција (*handler* би овде био празна функција, чији би позив само уносио непотребан трошак у мерење). Додатан разлог је избегавање дуплирања исте измерене променљиве: трошак позива функције преко показивача који је засебан предмет мерења код *FSM објеката стања* и *HSM имплементација*. *FSM структурног низа* са *handler*-ом помешао цену линеарне претраге са тим већ измереним трошком, уместо да тестира претрагу као чисту, независну променљиву.

2.4 Директни LUT FSM

Adamczyk описује *State Table Pattern* као решење намењено системима са великим бројем стања и транзиција, посебно у *realtime* и *embedded* окружењима где меморијски *overhead State Design Pattern*-а (заснованог на виртуелним функцијама) није прихватљив [PA06] . Контекст прослеђује догађај *Transition* класи која у константном времену враћа резултујуће стање. У одељку *Consequences*, аутор експлицитно наводи добре перформансе, $O(c)$ као предност, уз два недостатка: табела је скупа за конструкцију и тешка за одржавање, и носи висок иницијализациони *overhead*, који се, према аутору, може избећи коришћењем *compile-time* конструката (*struct*-ова) као елемената табеле. Имплементација у овом раду тачно то чини — табела транзиција је декларисана као *static const*, што значи да је иницијализована у време компајлирања, чиме избегава тај недостатак. Carlgren и Oskarsson описују структурно сродан приступ, који називају *Function Pointers*. Овај приступ представља дводимензиони низ показивача на *eventHandler* функције, чије димензије одређују број стања и број догађаја, индексирани тренутним стањем и догађајем (Слика 3).

```
1 ...
2 static eventHandler StateMachine =
3 {
4     [StateA] = {[Event1] = Event1Handler , [Event2] = Event2Handler},
5     [StateB] = {}
6 };
7 ...
8 nextState = (*StateMachine[currentState][newEvent])();
9 ...
```

Слика 3- Директни LUT Carlgren и Oskarsson [CO23]

Имплементација у овом раду има идентичан облик — индексирани дводимензиони низ `[state][event]` — али различит садржај: уместо показивача на функцију, свако поље директно чува вредност следећег стања (`transition_table[state][event]`) враћа

next_state директно, без иједног позива функције). Ово имплементацију чини структурно ближем *Function Pointers* приступу, а функционално ближем *State Table Pattern*-у, што значи да спаја облик једног извора са једноставношћу другог.

Директни LUT FSM имплементација у овом раду, приказана у кодном листингу испод (Кодни листинг 5), најтачније одговара Adamczyk-овом *State Table Pattern*-у. Исти механизам (директно индексирање унапред израчунате табеле, $O(c)$), иста предност (константно време), исти недостаци који су овде избегнути компајл-временском иницијализацијом. Облик табеле ($2D$ низ *[state][event]*) додатно се поклапа са структуром коју Carlgren и Oskarsson користе за свој *Function Pointers* приступ, уз једну суштинску разлику: њихова имплементација додаје позив функције који Директни *LUT FSM* у овом раду намерно изоставља, чиме добијамо чист тест $O(1)$ хипотезе — потврђене у експерименталном делу у наставку, кроз строго константан ($\min=\text{avg}=\max$) број циклуса у сопственим мерењима.

```
static const uint8_t transition_table[NUM_STATES][NUM_EVENTS] =
{
    /*                      EV_VALID          EV_INVALID
EV_TIMEOUT          */
    /* STATE_IDLE      */ { STATE_CHECKING, STATE_IDLE,
STATE_IDLE          },
    /* STATE_CHECKING */ { STATE_GRANTED,  STATE_DENIED,
STATE_CHECKING      },
    /* STATE_GRANTED   */ { STATE_GRANTED,  STATE_GRANTED,
STATE_IDLE           },
    /* STATE_DENIED    */ { STATE_DENIED,   STATE_DENIED,
STATE_IDLE           },
};

static inline uint8_t fsm_transition(uint8_t state, uint8_t
event) {
    return transition_table[state][event];
}
```

Кодни листинг 5-Директни LUT FSM имплементација

Кодни листинг 5 представља имплементацију коришћену у даљем раду, структурно аналогну генерисаном коду са Слика 3, уз изостављен позив *eventHandler* функције. Поље низа директно садржи вредност следећег стања.

2.5 FSM објеката стања

Adamczyk описује *Optimal FSM* образац [MS02] као решење за случајеве када је класичан објектно-оријентисани *State Design Pattern* (са виртуелним функцијама и посебном класом по стању) преспоро решење због превеликог броја индиректних позива. Ово је од посебног значаја за *embedded realtime* системе. Уместо да свако стање буде посебан објекат, стање се своди на показивач на функцију (метод), а "тренутно стање" јесте тај показивач [PA06] .

У одељку *Consequences*, Adamczyk наводи низ предности које се директно препознају у овој имплементацији: мали меморијски отисак ("*only one function pointer is required to implement the state machine*"), ефикасна транзиција ("*one assignment of state pointer*"), и једноставност имплементације. Кључна напомена о перформансама: "*Good performance — $O(\log n)$, where n is the number of cases in the switch statement*" — сам прелаз у ново стање је тренутан (једна додела), али обрада догађаја унутар *handler*-а зависи од броја грана којима се испитује који је догађај стигао, па укупна цена транзиције није чисто константна, већ зависи од те унутрашње структуре [PA06] .. Adamczyk експлицитно предлаже даљу оптимизацију. Замену тог грањања табелом претраге унутар критичних *handler*-а што је суштински Директни *LUT FSM* приступ примењен локално, унутар једне функције.

```
typedef uint8_t (*StateHandler)(uint8_t event);

static StateHandler const state_handlers[NUM_STATES] = {
    state_idle_handle,
    state_checking_handle,
    state_granted_handle,
    state_denied_handle,
};

static inline uint8_t fsm_transition(uint8_t state, uint8_t
event) {
    return state_handlers[state](event);
}
```

Кодни листина 6 – Директна LUT FSM имплементација

Сваки елемент кода са Кодни листинг 6 директно реализује по један део Adamczyk-овог описа. Низ *state_handlers[NUM_STATES]* представља реализацију идеје да се стање представи као метода — свако од четири стања је једна функција у овом низу (*state_idle_handle* је стање *IDLE*, не показивач ка неком посебном *IDLE* објекту). Променљива *current_state*, иако технички обично целобројно поље, представља показивач на тренутну методу коју Adamczyk описује: комбинација индекса и низа (ако је *current_state == 0*, активна метода је она на позицији 0, *state_idle_handle*) остварује исти ефекат као директан показивач на функцију, само индиректно, преко позиције у низу уместо преко сирове адресе. Позив *state_handlers[state](event)* у *fsm_transition()* затим остварује једну доделу показивача на стање коју Adamczyk наводи као предност — *handler*

функција враћа број следећег стања, који се потом уписује у *current_state* једном доделом без креирања или уништавања било каквог објекта.

Гранање по догађају, за разлику од избора стања, не дешава се у *fsm_transition()*, него унутар сваког појединачног *handler*-а — на пример, *state_checking_handle* садржи низ од две до три *if* провере којима се испитује који је конкретно догађај стигао, будући да је стање (а тиме и који ће се *handler* позвати) већ одређено пре самог позива. На тај унутрашњи број грана односи се Adamczyk-ова оцена $O(\log n)/O(n)$. Цена транзиције зависи од тога колико провера треба извршити унутар *handler*-а док се не пронађе поклапање са пристиглим догађајем. Ово директно објашњава зашто је FSM објекта стања имплементација у мерењима овог рада спорија од Директног LUT FSM-а (26 наспрам 17 циклус а на AVR платформи) упркос мањем меморијском заузећу. Директни LUT FSM овај корак гранања потпуно изоставља кроз директно читавање из унапред израчунате табеле, док FSM објекта стања мора, након што одабере одговарајући *handler*, још увек линеарно испитати који је догађај у питању.

Adamczyk суштински наводи да је овај образац јефтин по меморији, јер захтева само један низ показивача, али управо због тога није најбрже могуће решење. Ако је брзина приоритет, предлаже додавање табеле претраге (попут Директног LUT FSM приступа) унутар самог *handler*-а. Ова чињеница сугерише да се из овог дизајна не могу добити оба својства истовремено: или се остаје на малом меморијском отиску и прихвата $O(n)$ трошак грањања унутар *handler*-а, или се додаје табела и тиме троши више меморије да би се добила $O(1)$ брзина. Измерени резултати овог рада приказани у наставку (26 наспрам 17 циклуса, FSM објекта стања наспрам Директног LUT FSM-а) потврђују овај однос. Уштеда меморије FSM објекта стања приступа долази уз мерљиву цену у броју циклуса, у складу са оним што Adamczyk унапред описује.

2.6 Планарни HSM FSM

HSM (*Hierarchical State Machine* — хијерархијски коначни аутомат, у литератури познат и као *statechart*) представља проширење класичног (планарног) коначног аутомата у коме једно стање може да садржи комплетан под-аутомат са сопственим стањима и транзицијама. Концепт је први формално увео Harel (1987)

као решење за проблем комбинаторне експлозије стања код сложених система [DH87] . За разлику од планарног (*flat*) аутомата, где су сва стања равноправна и независна, HSM уводи однос родитељ-дете између стања: када је систем у неком детету (*substate*-у), он је истовремено логички и у његовом родитељу (*superstate*-у), што омогућава да се заједничко понашање више подстања дефинише једном, на нивоу родитеља, уместо да се понавља у сваком детету појединачно.

Adamczyk прецизно дефинише када овај образац има смисла, кроз услове применљивости који описују методологију овог рада: систем са много јефтиних транзиција на нивоу детета и мало скупљих транзиција на врху хијерархије [PA06] . Механизам који предлаже: "*Context object is the container of concrete state objects. Each concrete state class may have its own state machine for its child states.*" У делу *Consequences*, директно предвиђа и предност и цену: раздвајање на више нивоа чини сваки ниво једноставнијим, па су транзиције на сваком нивоу јефтиније за израчунавање — али додаје да, за разлику од угњеженог *switch-a* (где специјализација захтева преправку целе структуре), *Real-Time State Pattern* захтева измену само погођеног стања. Adamczyk не тврди да је хијерархија бржа од *flat* приступа, само да је лакша за проширивање [PA06] .

Тачно наслеђивање транзиција описали су Yasoub и Ammar у "*A Pattern Language of Statecharts*", 1998. Њихов концепт *transition inheritance* прецизно објашњава технику коју имплементација у овом раду користи: ако се у *superstate*-у дефинише транзиција на неки догађај, она важи за сва његова подстања, осим ако подстање дефинише сопствену, специфичнију транзицију за исти догађај — која тада има предност [YA98] .

Moreno описује аутомат за управљање точковима робота где је стање *Move* (кретање) дефинисано као родитељско стање које садржи подстања *Forward* и *Backward*. Аутор експлицитно наводи да ова хијерархија поједностављује транзицију ка стању *Stop*, која је дефинисана једном, на нивоу *Move*, и аутоматски важи и за *Forward* и за *Backward*, уместо да се понавља у оба [MV03] . Овај пример потврђује да је ова примена пронашла место и у роботичи.

Carlgren и Oskarsson имплементацију заснивају на приступу Sunithe и Samuela. Дефинишу *Context* структуру која чува активно стање и показивач на *dispatch* функцију која делегира догађаје ка одговарајућем стању, уз посебну *CompositeState* структуру која прати број региона и низове подстања за случајеве са више нивоа угњежености [SS19] . Ово је архитектонски тежа реализација истог концепта (укључује и подршку за *history* стања и *orthogonal* регионе, које имплементација у овом раду не користи). *HSM flat/nested* имплементације у овом раду представљају минималну реализацију исте идеје (*handler* + родитељ + резервисана вредност која означава необрађен догађај), без додатних механизма који овом експерименту нису потребни (*history* стање, паралелни региони).

Bubbling (пробубљавање) је понашање у току извршавања које се дешава сваки пут кад стигне догађај који дете не зна да обради. То се дешава при свакој транзицији која то захтева, током рада програма. Променљива *node* у коду само прати где се диспечер тренутно налази док тражи ко зна да одговори на догађај. На почетку је то увек право, тренутно стање аутомата (*node = state*). Ако *handler* тог стања не зна одговор, *node* се помери на његовог родитеља (*node = state_table[node].parent*, као што је приказано у коду - Кодни листинг 7), и провера се понавља тамо. Ово се дешава само унутар једног позива *fsm_transition()* — *node* се после мења и заборавља, док се стварно стање аутомата (*current_state*) не мења све док се не пронађе коначан одговор.

```
static const StateNode state_table[NUM_STATES] = {
    /* STATE_IDLE */ { state_idle_handle, -1
},
    /* STATE_CHECKING */ { state_checking_handle, -1
},
    /* STATE_DENIED */ { state_denied_handle, -1
},
    /* STATE_GRANTED */ { state_granted_handle, -1
},
    /* STATE_NORMAL_ACCESS */ { state_normal_access_handle,
STATE_GRANTED }, // child of GRANTED
    /* STATE_ADMIN_ACCESS */ { state_admin_access_handle,
STATE_GRANTED }, // child of GRANTED
};
static uint8_t fsm_transition(uint8_t state, uint8_t event) {
    int8_t node = state;
    uint8_t result;

    while (node != -1) {
        result = state_table[node].handler(event);
        if (result != EVENT_UNHANDLED) {
            return result;
        }
        node = state_table[node].parent;
    }
    return state;
}
```

Кодни листинг 7 – механизам bubbling-a

Имплементација у овом раду користи генерички диспечерски механизам (*handler* функција по стању + показивач на родитеља + резервисана вредност за необрађен догађај) над *FSM*-ом са истим скупом стања као у осталим тестираним приступима (*IDLE*, *CHECKING*, *GRANTED*, *DENIED*), али са празним пољем родитеља код сваког стања.

```
// ----- State table: every parent is -1 (NO real hierarchy
in this test) -----
static const StateNode state_table[NUM_STATES] = {
    { state_idle_handle,      -1 },
    { state_checking_handle,  -1 },
    { state_granted_handle,   -1 },
    { state_denied_handle,    -1 },
};

// ----- HSM dispatch: try current state, bubble up to
parent if unhandled -----
static uint8_t fsm_transition(uint8_t state, uint8_t event) {
    int8_t node = state;
    uint8_t result;

    while (node != -1) {
        result = state_table[node].handler(event);
        if (result != EVENT_UNHANDLED) {
            return result; // consumed by this state (or an
ancestor)
        }
        node = state_table[node].parent; // bubble up
    }
    return state; // nobody in the chain handled it -> stay in
current state
}
```

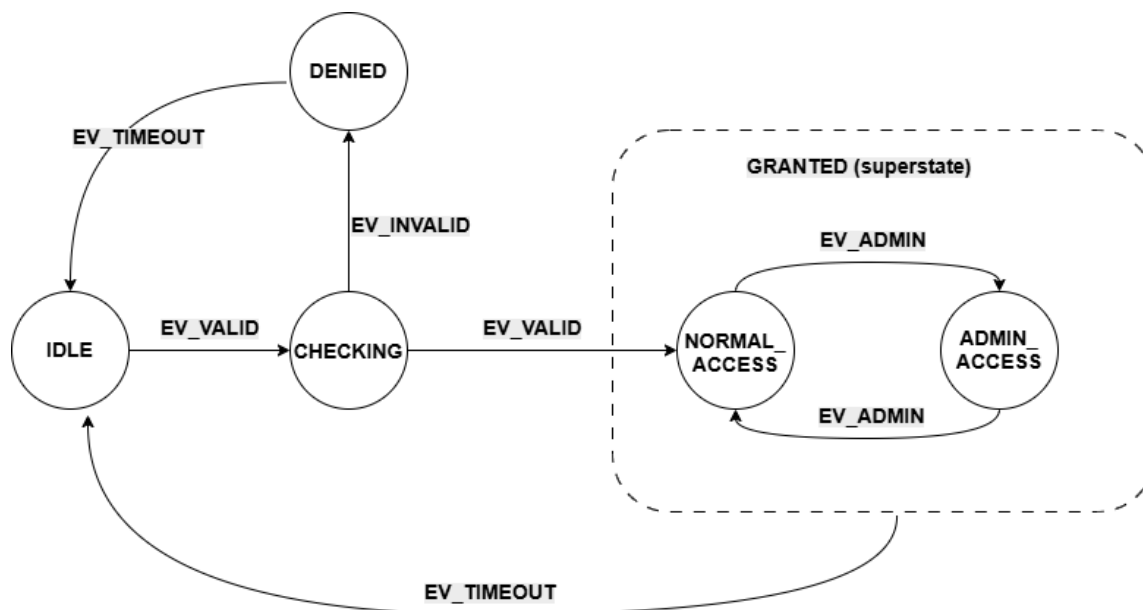
Кодни листинг 8 – Planarni HSM FSM

Кодни листинг 8 приказује *Планарну HSM FSM (HSM flat) имплементацију*. Пошто је *parent* свуда -1, диспетчерска петља (`while (node != -1)`) се у пракси никад не пење на други ниво. Сваки *handler* обрађује све своје догађаје сам, без потребе за пробубљавањем. Овај тест не мери корист од хијерархије (које овде нема), већ искључиво фиксни трошак самог диспетчерског механизма — петље и провере резервисане вредности — у поређењу са директним приступима попут Директног *LUT FSM*-а или *FSM* објеката стања. Планарни *HSM FSM* представља контролну тачку за Угњеждени *HSM FSM*. Разлика између њих изоловано показује цену стварног **пробубљавања**, одвојену од цене самог механизма.

2.8 Угњеждени HSM FSM

Угњеждени *HSM FSM* користи идентичан диспетчерски механизам описан у претходном одељку, али сада са стварним односом родитељ-дете између стања. Прва имплементација у овом раду код које **пробубљавање** описано у 2.6 стварно наступа. Скуп стања је проширен: *GRANTED* престаје да буде лист и постаје *superstate* који садржи два нова подстања, *NORMAL_ACCESS* и *ADMIN_ACCESS* (уведена је и нова врста догађаја, *EV_ADMIN*, која пребацује између њих). Ово

имплементација приказана на слици (Слика 4) реализована је по узору на Yasoub-Ammar-ова наслеђивања транзиција, која су описана у претходном одељку.



Слика 4 – Модификовани ХСМ ФСМ са родитељским стањем

Кодни листинг 9 приказује начин на који *state_granted_handle* обрађује искључиво *EV_TIMEOUT* — ову транзицију не дефинише ни *NORMAL_ACCESS* ни *ADMIN_ACCESS* појединачно, већ је наслеђују од заједничког родитеља *GRANTED*, на начин на који Yasoub и Ammar описују механизам. Свако од два подстања дефинише само своје специфично понашање (*EV_ADMIN*, пребацивање између *NORMAL_ACCESS/ADMIN_ACCESS*), док *EV_TIMEOUT* намерно остаје необрађен на том нивоу и пробубљава један корак више.

```

static uint8_t state_granted_handle(uint8_t event) {
    if (event == EV_TIMEOUT) return STATE_IDLE;
    return EVENT_UNHANDLED;
}
static uint8_t state_normal_access_handle(uint8_t event) {
    if (event == EV_ADMIN) return STATE_ADMIN_ACCESS;
    return EVENT_UNHANDLED;
}

```

Кодни листинг 9 - Угњеждени HSM FSM (HSM nested) имплементација

Методологија мерења директно циља овај случај: пре сваке мерене транзиције, стање се експлицитно поставља на *NORMAL_ACCESS*, а примењени догађај је увек *EV_TIMEOUT* — свака појединачна мерена транзиција тако изолије тачно један корак **пробубљавања**, без мешања са локално обрађеним транзицијама (*EV_ADMIN*). Разлика у броју циклуса између Планарног *HSM FSM*-а (без пробубљавања) и Угњежденог *HSM FSM*-а (један корак пробубљавања) тиме

представља чист, изолован трошак наслеђивања транзиција који литература описује, а овај рад у наставку експериментално доказује.

2.7 Угнеждени *LUT FSM* (Santic)

За разлику од осталих извора у овом раду, овај приступ потиче из практичног, инжењерског чланка намењеног *embedded* програмерима. Аутор Santic експлицитно поставља два начина имплементације FSM-а у C-у: угнеждене *switch* исказе (које назива једним од два уобичајена начина) и табелу претраге (*lookup table*), коју карактерише као концизнији приступ.

Коришћени Santic-ов механизам обухвата:

1. Набрајање свих стања и догађаја у енумерисаном типу података.
2. Креирање скупа табела, једна по стању, где сваки унос представља функцију која се извршава за тај пар (стање, догађај).
3. Постављање елемената табеле у јединствен дводимензиони низ индексиран стањем и догађајем.

Кључна разлика у односу на остале имплементације са показивачима на функције је што Santic-ове акционе процедуре типа *void* немају повратну вредност. Уместо да транзиција буде изражена кроз оно што функција врати, она се изражава кроз оно што функција уради као споредни ефекат, било да та акција представља подешавање новог стања унутар саме функције, Слика 5, [JSnd] .

```
void action_s1_e1 (void)
{
    /* do some processing here */

    current_state = STATE_2; /* set new state, if necessary */
}
```

Слика 5 - Santic акциона процедура без повратне вредности [JSnd]

Конкретно, акциона процедура сама, унутар тела функције, уписује ново стање у глобалну променљиву (*current_state = STATE_2;*), као што је приказано на слици (Слика 5), за разлику од имплементације са показивачима функција засноване на Carlgren и Oskarsson раду, где *handler* враћа следеће стање, а упис обавља позивалац споља (Слика 3). Број индиректних позива функције је идентичан у оба приступа (тачно један по транзицији) — разликује се само механизам преноса резултата (*return* вредност наспрам директног уписа дељене променљиве изнутра).

За разлику од Adamczyk-а и Carlgren-Oskarssona, који се фокусирају на перформансе и архитектуру, Santic експлицитно упозорава на безбедносни ризик специфичан за табеларне приступе. Угнеждени *switch* има уграђену заштиту (*default*: грана) која неважећи улаз једноставно игнорише или преусмери на

безбедно понашање. Директно индексирање низа (небитно да ли је Директни *LUT FSM* или било која варијанта са показивачима на функције) ту заштиту нема уграђену у сам механизам, и неопходно ју је експлицитно додати као проверу пре приступа низу. Иначе ће неважећи индекс довести до недефинисаног понашања (читање ван граница низа, потенцијални скок на произвољну адресу у меморији ако се ради о низу показивача на функције). Ово је корисна, практична напомена коју академски извори не помињу: брзина табеларних приступа има своју цену. Границе ручно мора да провери програмер, јер то механизам диспечовања не ради сам.

Имплементација ове методе у даљем раду преузима Santic-ов механизам у целости — и акционе процедуре без повратне вредности, и експлицитну проверу граница. То значи да се Santic-ов приступ од имплементације са показивачима на функције (Слика 5) разликује на два начина одједном: по томе како се преноси следеће стање (*return* вредност наспрам уписа изнутра), и по томе што има проверу граница коју она нема. Пошто су ова два фактора присутна заједно, из саме теорије не може се рећи колико сваки од њих доприноси евентуалној разлици у броју циклуса.

Зато се у експерименталном делу уводи и треће поређење — Директни *LUT FSM*, који нема ни позив функције ни проверу граница. Поређењем све три имплементације (Директни *LUT FSM*, имплементација са показивачима на функције, Santic-ов приступ) раздваја се колико од разлике у циклусима и заузећу меморије долази од самог позива функције, а колико додатно од провере граница [JSnd] .

3. Методологија мерења

У Поглављу 2 представљен је теоријски механизам сваког од осам разматраних имплементационих приступа, укључујући и хијерархијску организацију стања (*superstate/substate* однос, наслеђивање транзиција, *entry/exit* акције) на којој се заснивају Планарни и Угнежђени *HSM FSM*. У наставку је описан конкретан експериментални оквир у коме су сви приступи имплементирани и измерени: заједнички аутомат коришћен за поређење, коришћене хардверске платформе, и механизам мерења заузећа меморије и броја процесорских циклуса по транзицији.

3.1 Заједничка машина стања

Како би се различити стилови и архитектуре имплементације машина стања могли објективно упоредити, дефинисан је један заједнички, једноставан коначни аутомат који је затим имплементиран на осам различитих начина описаних у Поглављу 2 (Угнежђена и Модификована угнежђена *Switch-case FSM*, *FSM* структурног низа, Директни *LUT FSM*, *FSM* објеката стања, Планарни и Угнежђени *HSM FSM*, Индиректни *LUT FSM*), при чему су логика прелаза стања, скуп догађаја и мерна инфраструктура (бројач циклуса преко тајмера, *UART* комуникациони протокол и аутоматизована *benchmark* рутина од 1000 транзиција) идентична у свим приступима осим Планарног и Угнежђеног *HSM FSM*-а, који намерно уводе контролисану измену структуре ради изолованих мерења (детаљно образложено у одељцима 2.6 Планарни *HSM FSM* и 2.7 Угнежђени *LUT FSM* (Santic)). На овај начин свака разлика у заузећу меморије или броју извршених циклуса процесора може се приписати искључиво стилу односно архитектури имплементације, а не разлици у самој спецификацији понашања система.

Одабрани тест-пример представља поједностављени модел система контроле приступа (*Access Control FSM*), који моделује типичан сценарио провере пре него што се кориснику дозволи приступ неком ресурсу. Аутомат садржи четири стања:

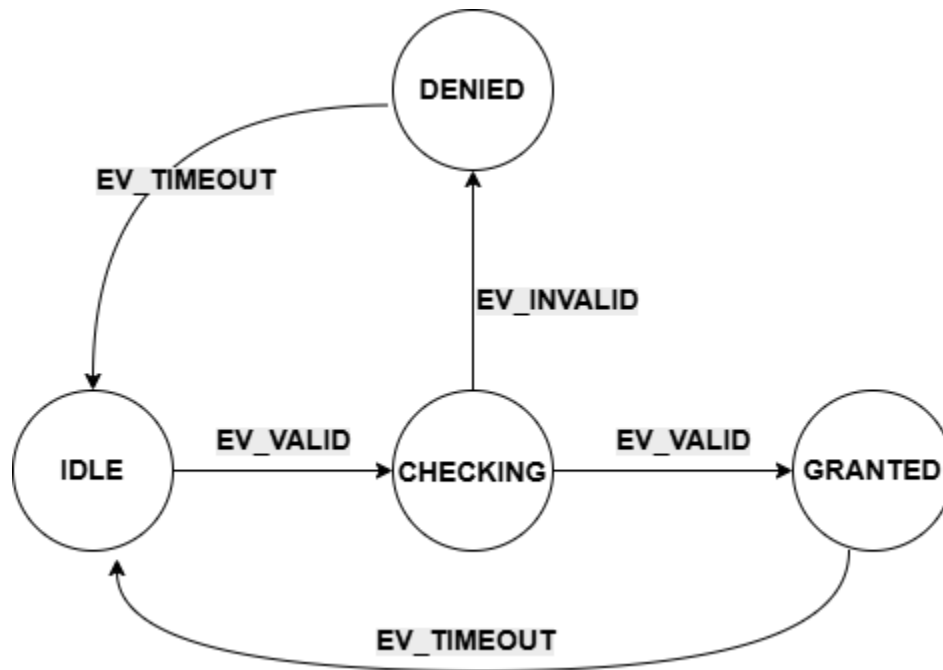
- **IDLE** — почетно стање у коме систем чека захтев за приступ
- **CHECKING** — прелазно стање у коме се врши провера ваљаности унетих акредитација
- **GRANTED** — стање у коме је приступ одобрен
- **DENIED** — стање у коме је приступ одбијен

Прелази између стања дефинисани су са три догађаја:

- **EV_VALID** — сигнализира да су акредитације препознате као исправне
- **EV_INVALID** — сигнализира да су акредитације препознате као неисправне
- **EV_TIMEOUT** — сигнализира истицање временског периода додељеног тренутном стању

Из стања *IDLE*, доласком догађаја *EV_VALID*, аутомат прелази у стање *CHECKING*. Из стања *CHECKING*, аутомат прелази у стање *GRANTED* уколико наступи догађај *EV_VALID*, односно у стање *DENIED* уколико наступи догађај

EV_INVALID. Из стања *GRANTED* и *DENIED*, аутомат се доласком догађаја *EV_TIMEOUT* враћа у почетно стање *IDLE*, чиме се затвара циклус и систем постаје спреман за нови захтев за приступ. Графички приказ аутомата дат је на слици (Слика 6).



Слика 6 - дијаграм стања контроле приступа FSM-а

Овај аутомат је сврсисходно одабран као минималан, али структурно репрезентативан пример: садржи линеаран низ прелаза (*IDLE* → *CHECKING*), грањање на основу исхода провере (*CHECKING* → *GRANTED* / *DENIED*) и повратне прелазе у почетно стање (*GRANTED* → *IDLE*, *DENIED* → *IDLE*), чиме покрива основне обрасце понашања који се јављају у знатно сложенијим системима, уз задржавање довољно мале величине (четири стања, три догађаја, пет дефинисаних прелаза) да омогући прегледну, ручно проверљиву анализу генерисаног машинског кода за сваку од разматраних имплементација.

Све имплементације су тестиране на платформи *Arduino Uno* (микроконтролер *ATmega328P*, чија је фреквенција 16 MHz), уз директан приступ хардверским регистрима (без коришћења *Arduino* библиотека попут *Serial*), како би се избегао додатни, тешко контролисан меморијски и временски *overhead* који би потекао од самог окружења, а не од тестиране *FSM* имплементације.

3.2 Хардверске платформе

Мерења су извршена на две архитектонски различите платформе: *Arduino Uno*, заснован на 8-битном микроконтролеру *ATmega328P* (AVR архитектура) који ради на фреквенцији од 16 MHz, и *ESP32*, заснован на 32-битном *Xtensa LX6* језгру које ради на фреквенцији од 240 MHz, коришћен преко *Arduino-ESP32 framework-a*. Ове две платформе представљају две различите тачке у распону ресурсно ограничених система. *ATmega328P* располаже свега 2 KB радне и 32 KB програмске меморије и извршава код *bare-metal*, без оперативног система, док *ESP32* располаже знатно већим ресурсима (~320 KB RAM-a) и извршава код под управљањем *FreeRTOS* оперативног система у позадини, што уводи додатне изворе недетерминизма приликом мерења (детаљно ће образложено у одељку 3.3 Методологија мерења циклуса).

За обе платформе, тестирани код приступа хардверским регистрима директно (без *Arduino* библиотека попут *Serial*), а комуникација са рачунаром ради читавања резултата мерења остварена је преко *UART* протокола на брзини од 9600 bps.

Архитектонска разлика између ова два микроконтролера огледа се и у начину организације меморије, што има директне последице на резултате мерења представљене у Поглављу 4. *ATmega328P* примењује строгу *Harvard* архитектуру — програмска (*Flash*) и радна (*SRAM*) меморија имају физички одвојене адресне просторе, тако да се подаци смештени у *Flash* меморији (нпр. константне табеле транзиција декларисане као *const*) не могу директно читати истим механизмом као подаци у *SRAM*-у, већ захтевају посебну инструкцију (*LPM*) или употребу *PROGMEM* директиве, у супротном се копирају у *SRAM* при покретању програма. *ESP32*, заснован на *Xtensa LX6* језгру, примењује модификовану *Harvard* архитектуру — на нивоу хардвера и даље постоје одвојене магистрале за инструкције и податке ради перформанси, али јединица за управљање меморијом (*Memory Management Unit, MMU*) мапира *Flash* меморију у јединствен, кеширан адресни простор доступан програму, чиме се константни подаци могу читати директно из *Flash-a* (*execute-in-place, XIP*), без потребе за копирањем у *RAM* или употребом посебних инструкција [ESnd] .

Ова разлика у меморијском моделу директно утиче на интерпретацију резултата у Поглављу 4. Идентичан изворни код, чија се величина статичке табеле мења, производи мерљиву *SRAM* разлику на *ATmega328P* платформи (услед строге поделе адресних простора), док на *ESP32* платформи иста промена остаје видљива искључиво у заузећу *Flash* меморије, не и *RAM-a*.

3.3 Методологија мерења циклуса

Поред заузећа меморијских ресурса, за анализе спроведене у овом раду од суштинске важности је и број циклуса процесора утрошених на прелазе између стања *FSM*-а. За рад са регистрима коришћеним у функцијама за мерење перформанси коришћен је званични *datasheet ATmega328P* микроконтролера.

3.3.1 Зашто није коришћен *ISR* приступ?

У овом раду није коришћен стандардан приступ заснован на прекидним рутинама (*Interrupt Service Routine*, *ISR*) за сам механизам мерења, јер такав приступ не би био довољно поуздан за мерење броја циклуса везаних за прелазе стања.

Прекидна рутина уводи неколико извора недетерминизма који су неприхватљиви када је циљ измерити тачан, поновљив број циклуса процесора утрошених на једну транзицију *FSM*-а:

- **Латенција уласка у прекид** — Од тренутка када се испуни услов за прекид до тренутка када процесор стварно почне извршавати код *ISR*-а пролази променљив број циклуса, јер процесор прво мора завршити тренутно извршавану инструкцију, а затим извршити интерну процедуру скока на вектор прекида и аутоматско чување стања (регистар *SREG*, повратна адреса на стеку). Ово кашњење само по себи уноси шум у мерење који нема везе са ценом саме *FSM* транзиције.
- **Могућност угнеждених или одложених прекида** — Ако се у тренутку окидања прекида процесор налази у другој, већ активној прекидној рутини, или је глобални бит прекида (*I-bit* у *SREG*) привремено искључен (нпр. унутар *cli()/sei()* блока), обрада прекида се одлаже за непознат, променљив број циклуса — што би мерење учинило непоновљивим.
- **Додатни трошкови уласка/изласка из *ISR*-а** — Сам улазак и излазак из прекидне рутине (чување и враћање регистара, извршавање инструкције *RET*) троши фиксан, али незанемарљив број циклуса који би се, уколико би се *ISR* користио за ограничавање мерног прозора, неизбежно помешао са бројем циклуса који се жели приписати искључиво функцији *fsm_transition()*.

Ова три извора недетерминизма директно су документована у техничкој спецификацији коришћеног микроконтролера [AMnd] . Према том документу, минимално време одзива на прекид износи четири циклуса такта, при чему се оно увећава уколико се прекид јави током извршавања вишециклусне инструкције, јер се та инструкција мора прво завршити пре него што се прекид опслужи — ово је извор варијабилне латенције уласка у прекид. Надаље, уласком у прекидну рутину бит за глобално омогућавање прекида (*I*-бит у регистру *SREG*) се аутоматски брише, чиме се сви наредни прекиди одлажу све док се тај бит експлицитно не постави — било инструкцијом *RET* при повратку из прекида, било ручно унутар саме рутине ради омогућавања угнежђених прекида — што уводи могућност непоновљивог кашњења обраде ако се прекиди временски поклопе. Коначно, сам повратак из прекидне рутине (инструкција *RET*) захтева фиксне четири циклуса такта, док регистар статуса *SREG* није аутоматски сачуван нити враћен при

уласку/изласку из прекида — то мора бити експлицитно урађено у софтверу, чиме се додатни циклуси троше на чување/враћање контекста који немају везе са мереном логиком.

Из ових разлога, мерење броја циклуса једне FSM транзиције захтева механизам чије је време окидања и трајање потпуно детерминистичко и синхрно са током главног програма — што *polling* приступ, за разлику од ISR-а, директно обезбеђује

3.3.2 Polling приступ коришћен за мерење

Уместо *ISR*-а, мерење се заснива на директном, синхронном читању вредности бројача тајмера (*polling*), непосредно пре и непосредно после позива функције *fsm_transition()*.

Овај приступ не генерише никакав прекид нити зависи од било каквог асинхроног догађаја — бројач тајмера се чита директно, у тренутку када то програм експлицитно затражи, чиме се елиминишу сви претходно наведени извори недетерминизма. Додатно, читав мерни прозор (позив *cycles_start()*, извршавање *fsm_transition()*, позив *cycles_stop()*) обавијен је паром *cli()/sei()* инструкција, које привремено онемогућавају све маскиране прекиде (брисањем глобалног I-бита у регистру *SREG*) за време трајања мерења. Тиме се спречава да евентуални други, неповезан прекид (нпр. *UART* пријем) прекине мерење усред извршавања и унесе додатне, нежељене циклусе у резултат.

ATmega328P располаже са три тајмера/бројача: 8-битним *TimerCounter0*, 16-битним *TimerCounter1* и 8-битним *TimerCounter2*. За потребе мерења броја циклуса по транзицији одабран је *TimerCounter1*, из следећих разлога:

- **Опсег бројача** — *TimerCounter1* је једини од три тајмера са пуном 16-битном ширином бројача (*TCNT1*, опсег 0–65535), док су *TimerCounter0* и *TimerCounter2* ограничени на 8 бита (*TCNT0/TCNT2*, опсег 0–255). Иако су појединачне *FSM* транзиције у разматраним имплементацијама довољно кратке (типично испод стотинак циклуса) да би стале и у 8-битни опсег, коришћење 16-битног бројача оставља резерву за имплементације са знатно више циклуса по транзицији, без ризика од прекорачења опсега бројања (*overflow*) током једног мерења.
- **Одвојеност од *Timer0*** — У делу анализе који испитује независност цене транзиције од дужине претходног боравка у стању, *Timer0* је наменски употребљен за симулацију трајања стања у *CTC* режиму рада уз прекидну рутину (*ISR(TIMER0_COMPA_vect)*), која генерише ~3-секундно кашњење пре него што се изазове мерена транзиција. Када би се за мерење циклуса транзиције користио исти тајмер који истовремено генерише ово кашњење, два механизма би делила исте регистре, што би захтевало пажљиво чување и обнављање конфигурације тајмера пре и после сваког мерења и унело додатни ризик од међусобног ометања. Коришћењем физички одвојене хардверске јединице (*TimerCounter1* за мерење циклуса,

TimerCounter0 за симулацију трајања стања) ова два механизма су потпуно независна.

3.3.3 Регистри *Timer1* коришћени за мерење

Регистар *TCCR1B* (*Timer/Counter1 Control Register B*) је 8-битни регистар који садржи три *Clock Select* бита, *CS12:CS11:CS10* (битови 2:1:0), који бирају извор такта за *Timer1* [AMnd] .

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{IO}/1$ (no prescaling)
0	1	0	$clk_{IO}/8$ (from prescaler)
0	1	1	$clk_{IO}/64$ (from prescaler)
1	0	0	$clk_{IO}/256$ (from prescaler)
1	0	1	$clk_{IO}/1024$ (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Слика 7 – Табела са вредностима прескалера за *Timer1* [AMnd]

Функција за почетак мерења (*cycles_start()*) прво поставља *TCCR1B* = 0, чиме се сви *clock-select* битови постављају на 0 и тајмер заустаља (нема такта, бројач се не помера). Затим се *TCNT1* = 0 уписује директно у *Timer/Counter1* бројачки регистар, ресетујући га на почетну вредност — пошто је *TCNT1* 16-битни регистар (мапиран преко *TCNT1H/TCNT1L* пара), *C* компајлер аутоматски генерише исправан двобајтни приступ (Кодни листинг 10).

```
static inline void cycles_start(void) {  
  
    TCCR1B = 0;  
    TCNT1 = 0;  
    TCCR1B = (1 << CS10);  
  
}
```

Кодни листинг 10 – Функција која означава почетак бројања циклуса помоћу регистра *Timer1*

На крају се *TCCR1B* = (1 << *CS10*) поставља *CS10* = 1, *CS11* = 0, *CS12* = 0, што означава *clk_{IO}* без прескалирања (Слика 7) — тајмер сада броји на пуној фреквенцији *CPU* такта (16 MHz), што значи да један *tick* одговара тачно једном *CPU* циклусу. Ово је кључно за мерење где бројач директно представља број протеклих *CPU* циклуса, без потребе за конверзијом.

```
static inline uint16_t cycles_stop(void) {

    TCCR1B = 0;
    return TCNT1;

}
```

*Кодни листинг 11 - Функција која означава завршетак бројања циклуса помоћу регистра *Timer1**

Кодни листинг 11 приказује функцију за крај мерења (*cycles_stop()*) и поново поставља *TCCR1B = 0*, чиме се тајмер зауставља и спречава да бројач настави да расте док се вредност чита или док се извршава неки други код. На крају, *return TCNT1* чита тренутну вредност 16-битног бројача и враћа је као резултат — пошто је тајмер стартован на 0 и бројао без прескалирања тачно онолико циклуса колико је прошло између *cycles_start()* и *cycles_stop()*, ова вредност јесте број *CPU* циклуса који су протекли у том интервалу.

3.3.4 Заједничка функција за мерење транзиције

Функција *measured_transition()* представља заједничку обавијајућу функцију (*wrapper function*) која се користи у свих осам имплементација и чији је задатак да изолује тачан број циклуса процесора утрошених на позив функције *fsm_transition()* — конкретна имплементација ове функције разликује се од примера до примера (угнеждени *switch*, линеарна претрага, индексирана табела, диспечовање преко показивача на функцију или *HSM* механизам са **пробубљавањем**), док логика око ње остаје непромењена.

```
static uint16_t measured_transition(uint8_t event) {

    cli();
    cycles_start();
    uint8_t next = fsm_transition(current_state, event);
    uint16_t cycles = cycles_stop();
    sei();
    current_state = next;
    return cycles;

}
```

Кодни листинг 12 – заједничка функција за мерење броја циклуса потребних за прелазе између стања

Пре позива *fsm_transition()* извршава се инструкција *cli()*, која брише глобални бит за омогућавање прекида (I-бит у регистру *SREG*) и тиме онемогућава све маскирајуће прекиде за време трајања мерног прозора, спречавајући да неки

неповезан прекид унесе додатно, променљиво кашњење у мерење. Након завршетка мерења позива се *sei()*, која тај бит поново поставља и враћа прекидни систем у нормалан рад. Између ова два позива налазе се *cycles_start()* и *cycles_stop()*, које мере тачан број протеклих циклуса око позива *fsm_transition()*, док се ажурирање глобалног стања (*current_state = next*) намерно извршава тек након што је мерење већ завршено, како не би утицало на измерену вредност.

3.4 Нивои оптимизације компајлера

Према званичној GCC документацији [GNUnd] и стандардној литератури за микроконтролере, нивои оптимизације представљају унапред дефинисане скупове оптимизационих пролаза компајлера. Кроз ове флегове компајлер прави компромис између три фактора: брзине извршавања, заузећа меморије (*Flash/RAM*) и могућности дебаговања.

-O0 — Без оптимизације (*No optimization / Debugging baseline*)

Ово подешавање искључује практично све оптимизационе алгоритме у компајлеру. Циљ је најбрже могуће време превођења (*compilation time*) и очување тачне структуре кода ради једноставног дебаговања преко GDB-а или хардверских емулятора. Свака променљива се строго чува и чита са стека или из *RAM*-а, а редослед инструкција се 1:1 поклапа са изворним C кодом. Утицај на микроконтролер (AVR): знатно веће заузеће *Flash* меморије и приметно већи број циклуса, односно спорије извршавање.

-Os — Оптимизација за величину (*Optimize for size*)

Ово је подразумевани ниво оптимизације за већину *embedded* платформи, укључујући Arduino core [MCnd]. Укључује све оптимизације из -O2 које не повећавају величину бинарног кода. Циљ је минимално заузеће *Flash* и програмског простора на меморијски ограниченим микроконтролерима. Искључују се технике попут агресивног одмотавања петљи (*loop unrolling*), поравнања функција и блокова (*function/loop alignment*) или агресивног уграђивања функција (*inlining*) уколико оне генеришу више бајтова машинског кода. Утицај на микроконтролер (AVR): најмањи *.hex* фајл и најмање заузеће програмске меморије уз веома добре брзинске перформансе.

-O2 — Оптимизација за перформансе / брзину (*Moderate-to-high speed optimization*)

Ово подешавање укључује готово све подржане оптимизације које не подразумевају превелики компромис у величини кода. Циљ је максимално убрзање рада процесора и смањење броја циклуса по инструкцији/функцији. Активирају се напредни пролази попут елиминације заједничких подизраза (*Global Common Subexpression Elimination*, GCSE), анализе тока података, елиминације мртвог кода и оптимизације скокова и грањања (*thread jumps, crossjumping*). Утицај на микроконтролер (AVR): убрзава прелазе стања у *FSM*-у и смањује број циклуса, али често резултује нешто већим *Flash* бинарним фајлом у односу на -Os.

4. Резултати мерења и дискусија резултата

4.1 Резултати на AVR платформи

Сва мерења у овом одељку изведена су на платформи Arduino Uno (*ATmega328P*, 16 MHz), према методологији описаној у Поглављу 3 Методологија мерења, за осам именованих приступа уведених у Поглављу 2 Теоријске основе.

Начин имплементације	Flash - O0	Flash - Os	Flash - O2	SRAM -O0	SRAM -Os	SRAM -O2
Модификовани угњеждени <i>switch-case FSM</i>	2512 bytes	1476 bytes	1928 bytes	286 bytes	290 bytes	290 bytes
Угњеждени <i>switch-case FSM</i>	2582 bytes	1476 bytes	1928 bytes	286 bytes	290 bytes	290 bytes
<i>FSM</i> пиза структура	2518 bytes	1494 bytes	1952 bytes	304 bytes	308 bytes	308 bytes
Директни <i>LUT FSM</i>	2438 bytes	1526 bytes	1978 bytes	298 bytes	302 bytes	302 bytes
Индиректни <i>LUT FSM (Santic)</i>	2642 bytes	1568 bytes	2090 bytes	340 bytes	346 bytes	346 bytes
<i>FSM</i> објеката стања	2562 bytes	1520 bytes	2008 bytes	292 bytes	298 bytes	298 bytes
Планарни <i>HSM FSM</i>	2640 bytes	1564 bytes	2024 bytes	302 bytes	308 bytes	308 bytes
Угњеждени <i>HSM FSM</i>	2796 bytes	1688 bytes	2048 bytes	410 bytes	418 bytes	418 bytes

Табела 1- Резултати мерења, заузеће Flash/SRAM меморије код *ATMega328P* микроконтролера

Заузеће радне меморије (SRAM) остаје стабилно за сваку имплементацију независно од нивоа оптимизације, док заузеће Flash меморије опада са -O0 на -Os, а затим благо расте на -O2. Овај образац је доследан кроз свих осам приступа.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угњеждени <i>switch-case FSM</i>	78	81	84
Угњеждени <i>switch-case FSM</i>	90	92	95
<i>FSM</i> пиза структура	121	160	211
Директни <i>LUT FSM</i>	79	79	79
Индиректни <i>LUT FSM (Santic)</i>	82	82	83
<i>FSM</i> објеката стања	105	105	105
Планарни <i>HSM FSM</i>	134	134	134
Угњеждени <i>HSM FSM</i>	213	213	213

Табела 2 – Мерења саоптимизацијом -O0

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угњеждени <i>switch-case FSM</i>	13	13	16
Угњеждени <i>switch-case FSM</i>	13	13	16
<i>FSM</i> пиза структура	25	42	64
Директни <i>LUT FSM</i>	17	17	17
Индиректни <i>LUT FSM (Santic)</i>	38	38	39
<i>FSM</i> објеката стања	26	26	26
Планарни <i>HSM FSM</i>	37	37	39
Угњеждени <i>HSM FSM</i>	72	72	72

Табела 3 - Мерења саоптимизацијом -Os

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угњеждени <i>switch-case FSM</i>	13	13	16
Угњеждени <i>switch-case FSM</i>	13	13	14
<i>FSM</i> пиза структура	25	39	59
Директни <i>LUT FSM</i>	18	18	18
Индиректни <i>LUT FSM (Santic)</i>	37	37	38
<i>FSM</i> објекта стања	24	24	25
Планарни <i>HSM FSM</i>	36	37	39
Угњеждени <i>HSM FSM</i>	71	71	71

Табела 4 - Мерења саоптимизацијом -O2

4.1.1 Разлика између стилова писања кода – *return* наспрам *break*

На нивоу -O0, Угњеждена *Switch-case FSM (break)* заузима 2582 бајта програмске меморије, док Модификована угњеждена *Switch-case FSM (return)* заузима 2512 бајтова — разлика од 70 бајтова (2,71%), искључиво у *Flash* меморији (*SRAM* идентичан, 286 бајтова). На -Os и -O2, ова разлика у потпуности нестаје (идентичних 1476/1928 бајта, и идентичан број циклуса на оба нивоа осим у максималним вредностима) — последица елиминације мртвих уписа (*dead store elimination*) коју GCC примењује на овим нивоима, чиме се две синтаксно различите, семантички еквивалентне имплементације свODE на идентичан облик пре генерисања машинског кода.

4.1.2 Директни *LUT FSM* — потврда константне временске сложености

Директни *LUT FSM* показује идентичних 17 циклуса у сва три статистичка показатеља на -Os (18 на -O2) — потпуна одсутност варијације директно потврђује теоријску $O(c)$ тврдњу коју смо у одељку 2.4 повезале са Adamczyk-овим State Table Pattern-ом. За разлику од овога, *FSM* структурног низа показује мерљив распон (25–64 циклуса на -Os), јер број извршених поређења зависи од позиције тражене транзиције у низу.

4.1.3 FSM објеката стања — компромис меморије и брзине

FSM објеката стања је истовремено меморијски јефтинији (1520 наспрам 1526 бајта на

-Os) и спорији (26 наспрам 17 циклуса) од Директног *LUT FSM-a* — потврђује Adamczyk-ову $O(\log n)/O(n)$ оцену из одељка 2.5.

4.1.4 Хијерархијске имплементације — најскупљи приступ на сваком нивоу

Угњеждени *HSM FSM* показује највеће заузеће *Flash* меморије на сва три нивоа оптимизације (2796/1688/2048 В) и истовремено највећи број циклуса (213/72/71), чиме доследно заузима последње место по оба критеријума. Планарни *HSM FSM* (контролна варијанта без стварне хијерархије) показује знатно мање циклусе (134/37/37) — разлика између ове две имплементације (нпр. 72 наспрам 37 на -Os, готово дупло) представља изолован трошак једног корака **пробубљавања**, потврђен емпиријски.

4.1.5 Индиректни LUT FSM (Santic) — трошак провере граница

Santic *LUT* показује 82/38/37 циклуса — спорији од Директног *LUT FSM-a* (17/18), али у истом опсегу као *FSM* структурног низа. Детаљно разлагање овог трошка на компоненту саме индирекције и компоненту провере граница дато је у одељку 4.5.

4.2 Поређење AVR резултата са литературом

Резултати приказани у Табелама 1– 4 могу се систематски упоредити са налазима из два независна извора: Carlgren и Oskarsson и Adamczyk-овим каталогом образаца пројектовања.

Поклапање је најизраженије за хијерархијске имплементације. Угњеждени *HSM FSM* у приложеним табелама показује највеће заузеће програмске меморије на сва три испитана нивоа оптимизације и истовремено највећи број циклуса процесора по транзицији, чиме доследно заузима последње место по оба критеријума. Овај налаз директно потврђује закључак Carlgren-Oskarssonovog рада, у коме *Hierarchical State Pattern* имплементација „доследно показује највеће заузеће меморије... и најлошије резултате у погледу времена извршавања и скалабилности са порастом броја стања" [CO23] , као и Adamczyk-ов опис Real-Time State Pattern-a, према коме механизам подршке за угнежђена стања уводи додатну структурну сложеност „чак и када се та додатна структура у конкретном случају не искоришћава у пуној мери" [PA06] .

Додатно поклапање уочено је код Директног *LUT FSM-a*, чији је број циклуса процесора идентичан у сва три статистичка показатеља на сваком испитаном нивоу оптимизације, чиме се потврђује теоријска тврдња о константној временској сложености $O(c)$ коју Adamczyk приписује *State Table Pattern-u* [PA06] .

На нивоу -O2, налаз да Угњеждена *Switch-case FSM* захтева најмање програмске меморије поклапа се са *case-study* резултатом из рада Carlgren и Oskarsson, где се наводи да „the Nested Switch implementation needs the least amount of memory” при истом нивоу оптимизације [CO23] .

Два налаза се, међутим, разилазе од литературе. Прво, на нивоу -O0, *FSM* структурног низа не показује ни најмање заузеће меморије ни најкраће време извршавања — трећи је по оба критеријума, иза Угњеждене *Switch-case FSM* и Директног *LUT FSM*-а — док Carlgren и Oskarsson у свом *case-study*-ју без оптимизационих флегова наводе супротно [CO23] . Вероватно објашњење лежи у разлици величине испитиваног аутомата: Carlgren и Oskarsson рад испитује генерисане аутомате са до 1000 стања, док аутомат коришћен у овом раду има свега четири стања и пет дефинисаних транзиција — на тој величини фиксни трошак петље за линеарну претрагу надмашује уштеду коју би та претрага иначе донела.

Друго, *HSM* имплементације у овом раду захтевају више програмске меморије од Директног *LUT FSM*-а на сва три нивоа оптимизације, док Carlgren и Oskarsson рад у општем делу поређења наводи да табеларни приступи захтевају највише меморије од свих шест испитаних приступа [CO23] . Ова разлика се такође може приписати величини аутомата: заузеће меморије табеларног приступа расте пропорционално производу броја стања и броја догађаја ($O(N \times M)$), док фиксни трошак хијерархијске инфраструктуре расте спорије, приближно линеарно са бројем стања.

4.3 Резултати на ESP32 платформи

Недостаје ми Santic pristup ovde, tj taj kod nisam pokrenula i nedostaje mi da pokrenem ‘prazne prog’ da vidimo zauzece kako bi rezultati bili poredivi, a ne kao u prvom dokumentu u hiljadama bajtova.

4.4 Упоредна анализа AVR и ESP32

Ovo poglavlje cu uraditi u nedelju kada budem formirala rezultate da budu smisleni brojevi ya neku vrstu poredjenja 😊

4.5 Разлагање трошка индиректних приступа

Литература

[PA06] Adamczyk, P. *The Anthology of the Finite State Machine Design Patterns*, 2006.

[CO23] Carlgren, J., Oskarsson, P. W. *State Machine Model-To-Code Transformation In C*. UPTec F 23044, Uppsala University, 2023.

[YA98] Yacoub, S., Ammar, H. *A Pattern Language of Statecharts*. 1998.

[MV03] Moreno, A., Valduvieto, J. *dFSM: Finite State Machines for Embedded Systems*. Real-Time Linux Workshop (RTLWS), 2003.

[KJH23] Kadam, S., Jogalekar, S., Hembade, S. *Model a Finite State Machine as a Construct in Computer Programming*. Journal of Data Acquisition and Processing, Vol. 38 (1), 2023.

[DH87] Harel, D. *Statecharts: A Visual Formalism for Complex Systems*. Science of Computer Programming, Vol. 8, str. 231–274, 1987.

[JSnd] Santic, J. *Writing Efficient State Machines in C*. Dostupno na: <http://johnsantic.com/comp/state.html> (pristupljeno 2026).

[AK17] Kumar, A. *How to implement finite state machine in C*. Articleworld, 13. avgust 2017. Dostupno na: <https://articleworld.com/state-machine-using-c/>

[MS02] Samek, M. *Practical Statecharts in C/C++*. CMP Books, 2002.

[SS19] Sunitha, E. V., Samuel, P. *Automatic Code Generation From UML State Chart Diagrams*. IEEE Access, 7:8591–8608, 2019.

[ESnd] Espressif Systems. *ESP-IDF Programming Guide — Memory Types (ESP32)*. Dostupno na: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/memory-types.html> (pristupljeno 2026).

[ES1nd] Espressif Systems. *ESP-IDF Programming Guide — Memory Types (ESP32)*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/memory-types.html> (pristupljeno 2026).

[ES2nd] Espressif Systems. *ESP-IDF Programming Guide — ESP-IDF FreeRTOS SMP Changes*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/freertos-smp.html> (pristupljeno 2026).

[AMnd] Atmel / Microchip. *ATmega328P Datasheet Complete*. Dostupno na: https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328_P%20AVR%20MCU

[%20with%20picoPower%20Technology%20Data%20Sheet%2040001984A.pdf](#)
(приступљено 2026).

[GNUnd] GNU Project. *Using the GNU Compiler Collection — Options That Control Optimization*. Одељак 3.10. Доступно на: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> (приступљено 2026).

[MCnd] Microchip Technology. *AVR-GCC Optimization Guide* /*Nadji nadja link gde si ga nasla*